

CSE221: Algorithms

Breadth First Search

Prepared by: A B M Muntasir Rahman [MUNR]



Inspiring Excellence

Graph Traversal

- Given a graph $G = (V, E)$, and a source vertex s , we want to visit all the vertices $\in V$ exactly once!
- Two graph traversal algorithms: Breadth First Search (BFS) and Depth First Search (DFS)

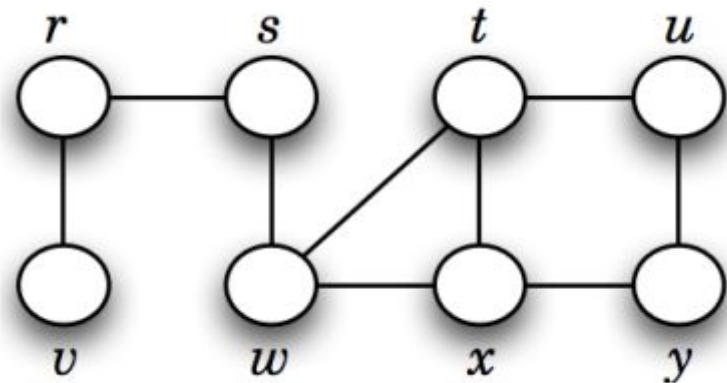
Breadth First Search (BFS)

- Traverses the entire graph level-wise.
- Uses **Queue** for traversal.

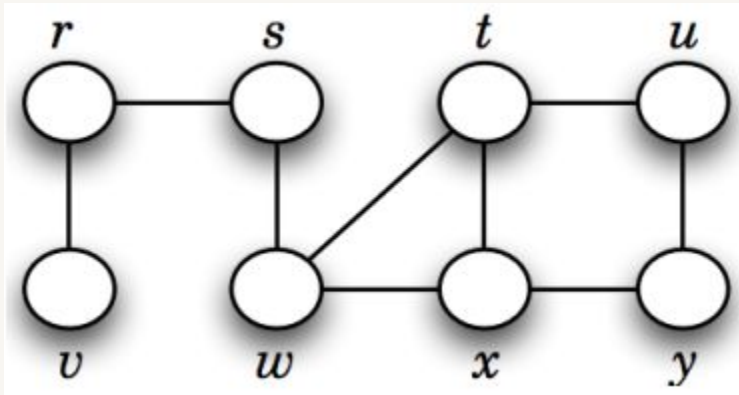
If we start from source vertex *s*,

then first level vertices are *r*, *w*
second level vertices are *v*, *t*, *x*
third level vertices are *u*, *y*

So, **BFS traversal** order will be,
s, *r*, *w*, *v*, *t*, *x*, *u*, *y*



Simulation



Visited

0	0	0	0	0	0	0	0
<i>r</i>	<i>s</i>	<i>t</i>	<i>u</i>	<i>v</i>	<i>w</i>	<i>x</i>	<i>y</i>

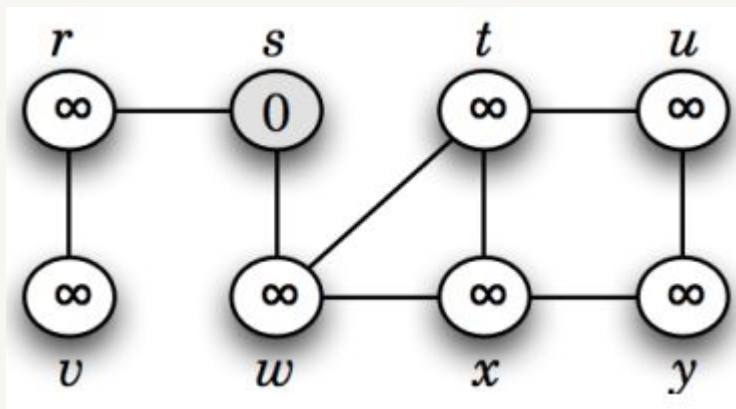
Distance

0	0	0	0	0	0	0	0
<i>r</i>	<i>s</i>	<i>t</i>	<i>u</i>	<i>v</i>	<i>w</i>	<i>x</i>	<i>y</i>

Parent

N	N	N	N	N	N	N	N
<i>r</i>	<i>s</i>	<i>t</i>	<i>u</i>	<i>v</i>	<i>w</i>	<i>x</i>	<i>y</i>

Simulation



Q

s
0

Visited

0	1	0	0	0	0	0	0
r	s	t	u	v	w	x	y

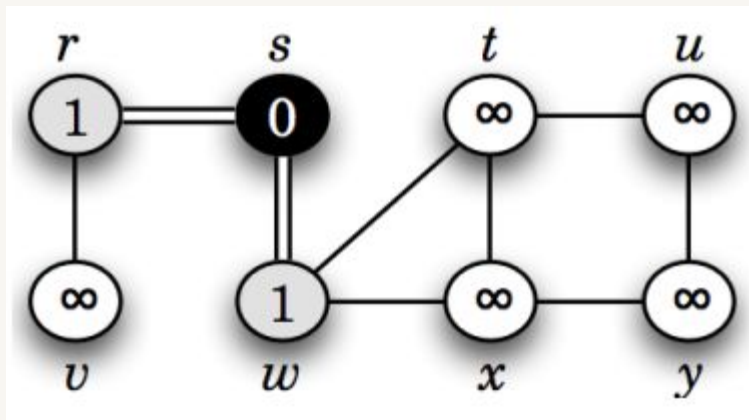
Distance

0	0	0	0	0	0	0	0
r	s	t	u	v	w	x	y

Parent

N	-1	N	N	N	N	N	N
r	s	t	u	v	w	x	y

Simulation



Q

w	r
1	1

Visited

1	1	0	0	0	1	0	0
r	s	t	u	v	w	x	y

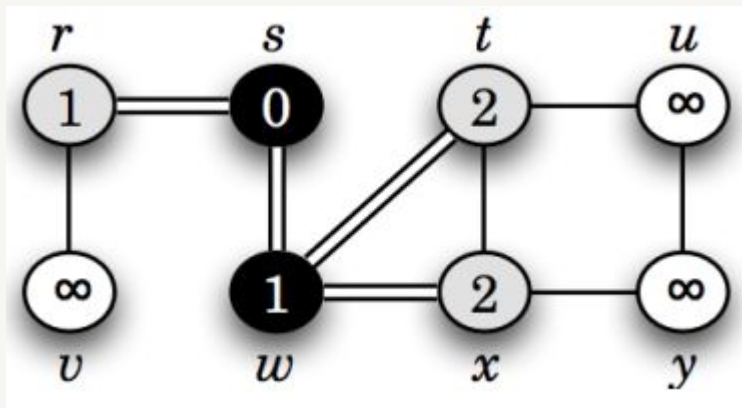
Distance

1	0	0	0	0	1	0	0
r	s	t	u	v	w	x	y

Parent

s	-1	N	N	N	s	N	N
r	s	t	u	v	w	x	y

Simulation



Q

r	t	x
1	2	2

Visited

1	1	1	0	0	1	1	0
r	s	t	u	v	w	x	y

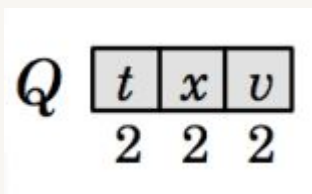
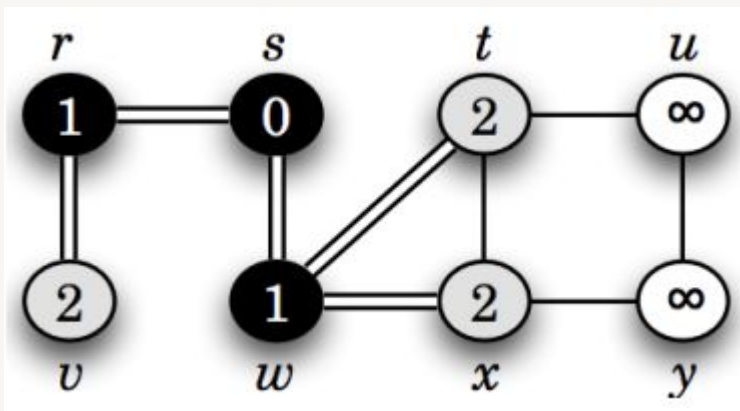
Distance

1	0	2	0	0	1	2	0
r	s	t	u	v	w	x	y

Parent

s	-1	w	N	N	s	w	N
r	s	t	u	v	w	x	y

Simulation



Visited

1	1	1	0	1	1	1	0
r	s	t	u	v	w	x	y

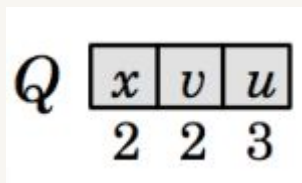
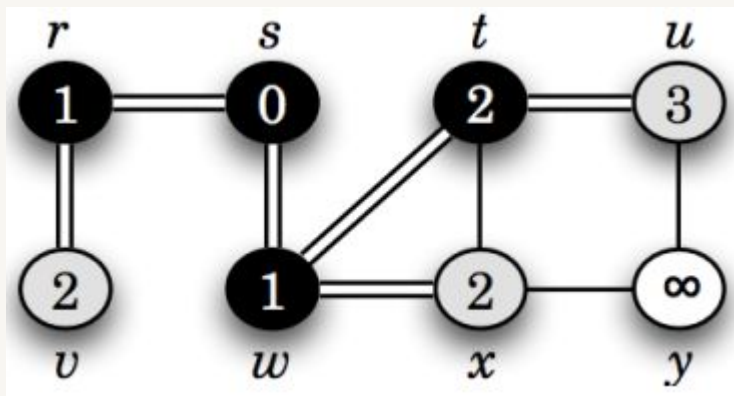
Distance

1	0	2	0	2	1	2	0
r	s	t	u	v	w	x	y

Parent

s	-1	w	N	r	s	w	N
r	s	t	u	v	w	x	y

Simulation



Visited

1	1	1	1	1	1	1	0
r	s	t	u	v	w	x	y

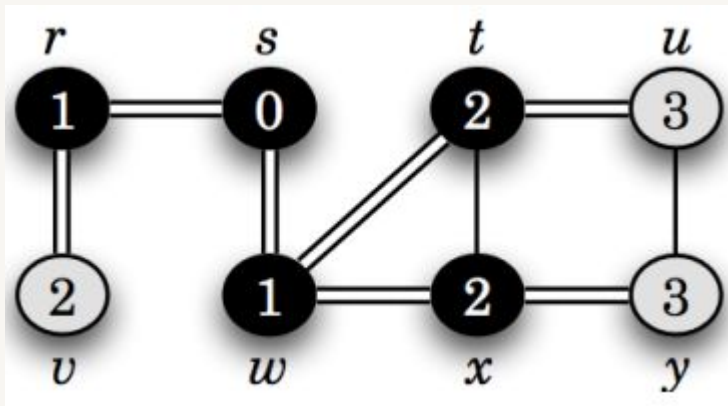
Distance

1	0	2	3	2	1	2	0
r	s	t	u	v	w	x	y

Parent

s	-1	w	t	r	s	w	N
r	s	t	u	v	w	x	y

Simulation



Q

v	u	y
2	3	3

Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

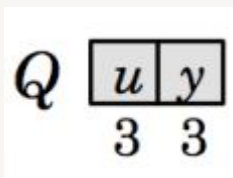
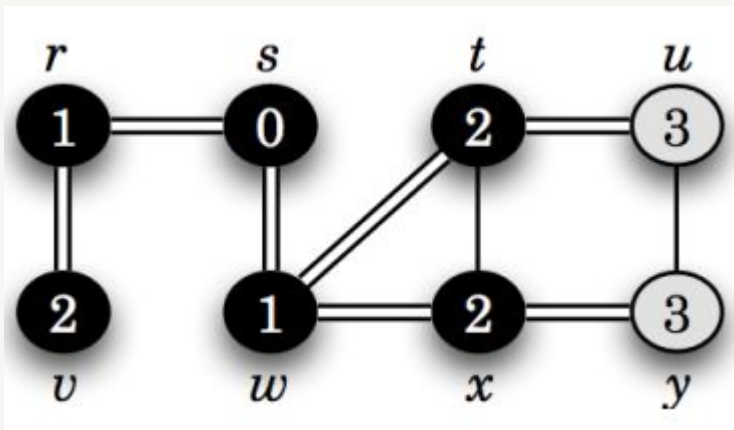
Distance

1	0	2	3	2	1	2	3
r	s	t	u	v	w	x	y

Parent

s	-1	w	t	r	s	w	x
r	s	t	u	v	w	x	y

Simulation



Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

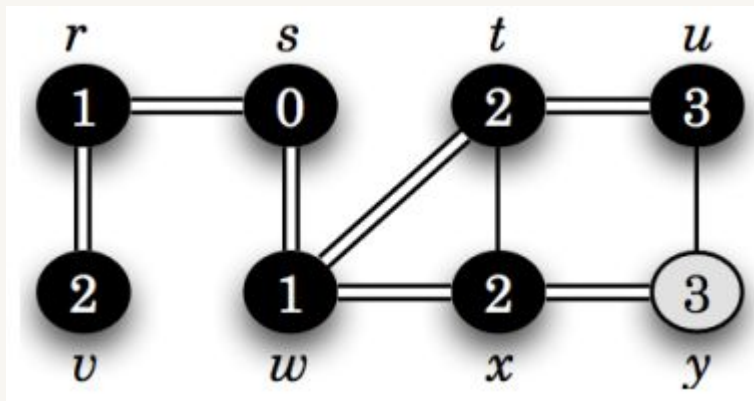
Distance

1	0	2	3	2	1	2	3
r	s	t	u	v	w	x	y

Parent

s	-1	w	t	r	s	w	x
r	s	t	u	v	w	x	y

Simulation



Q *y*
3

Visited

1	1	1	1	1	1	1	1
<i>r</i>	<i>s</i>	<i>t</i>	<i>u</i>	<i>v</i>	<i>w</i>	<i>x</i>	<i>y</i>

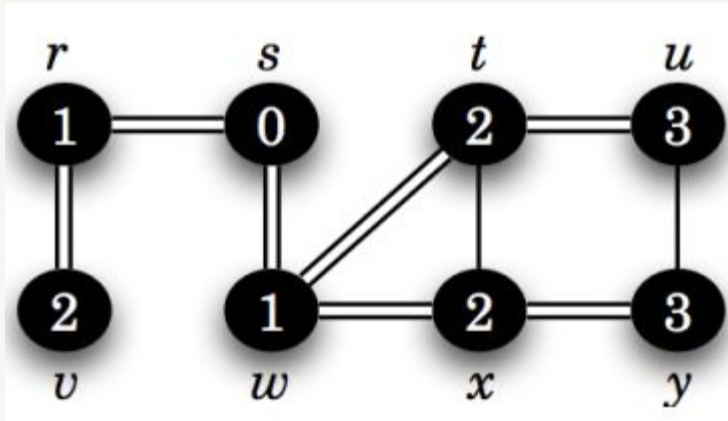
Distance

1	0	2	3	2	1	2	3
<i>r</i>	<i>s</i>	<i>t</i>	<i>u</i>	<i>v</i>	<i>w</i>	<i>x</i>	<i>y</i>

Parent

<i>s</i>	-1	<i>w</i>	<i>t</i>	<i>r</i>	<i>s</i>	<i>w</i>	<i>x</i>
<i>r</i>	<i>s</i>	<i>t</i>	<i>u</i>	<i>v</i>	<i>w</i>	<i>x</i>	<i>y</i>

Simulation



$Q \emptyset$

Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

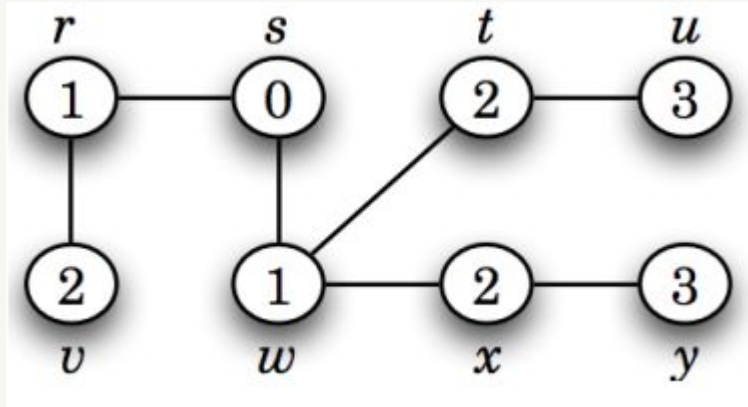
Distance

1	0	2	3	2	1	2	3
r	s	t	u	v	w	x	y

Parent

s	-1	w	t	r	s	w	x
r	s	t	u	v	w	x	y

Simulation



BFS traversal sequence will be,

s, r, w, v, t, x, u, y

Simulation

Visited

1	1	1	1	1	1	1	1
r	s	t	u	v	w	x	y

What is the shortest distance between s to u?

Ans: Go to index u of distance array (3)

Distance

1	0	2	3	2	1	2	3
r	s	t	u	v	w	x	y

What is the shortest path from s to u?

Ans: Go to index u of parent array (t). Since t is not source, go to index t (w). w is not the source, so go to index w (s). Since s is source, path will be s -> w -> t -> u.

Parent

s	-1	w	t	r	s	w	x
r	s	t	u	v	w	x	y

BFS Pseudocode

BFS(G, s)

$Q = []$, $visited = []$, $parent = []$, $distance = []$

$visited[s] = 1$, $parent[s] = -1$, $distance[s] = 0$

ENQUEUE(Q, s)

while Q is not empty

$u = \text{DEQUEUE}(Q)$ //Dequeued sequence is the BFS sequence

 for each vertex $v \in \text{Adj}[u]$

 if $visited[v] = 0$

$visited[v] = 1$

$distance[v] = distance[u] + 1$

$parent[v] = u$

 ENQUEUE(Q, v)

BFS Properties

- Can find the shortest path and minimum distance required to traverse from a source vertex to all other vertices in an unweighted graph.
- Can always find the solution.
- In case of directed graph, not all vertices may be explored.
- Not suitable for detecting cycles in a graph.

Time Complexity: $O(n + m)$

Space Complexity: $O(n + m)$

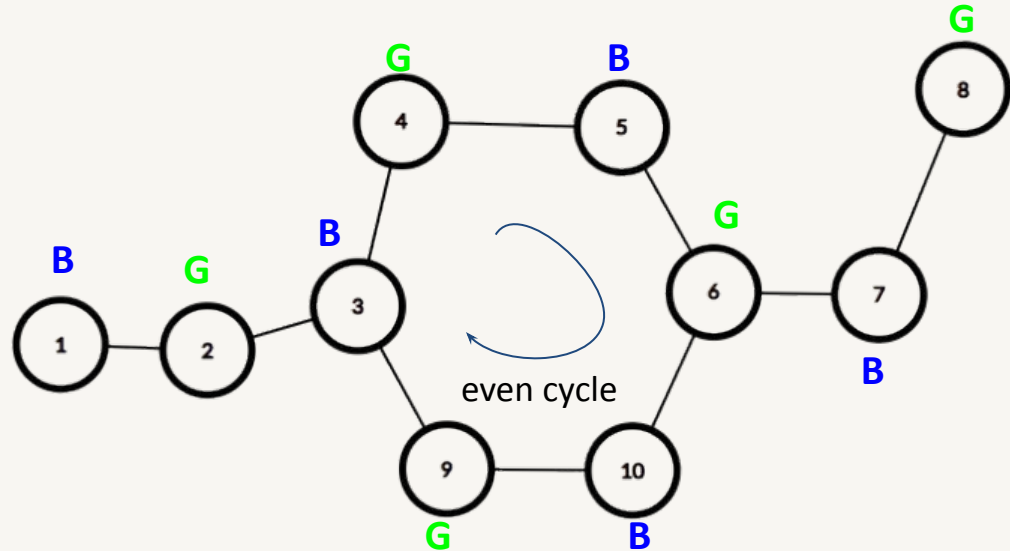
Bipartite/Bicolorable Graph

- A graph that can be colored using exactly two colors
- However, no two adjacent vertices should have the same color

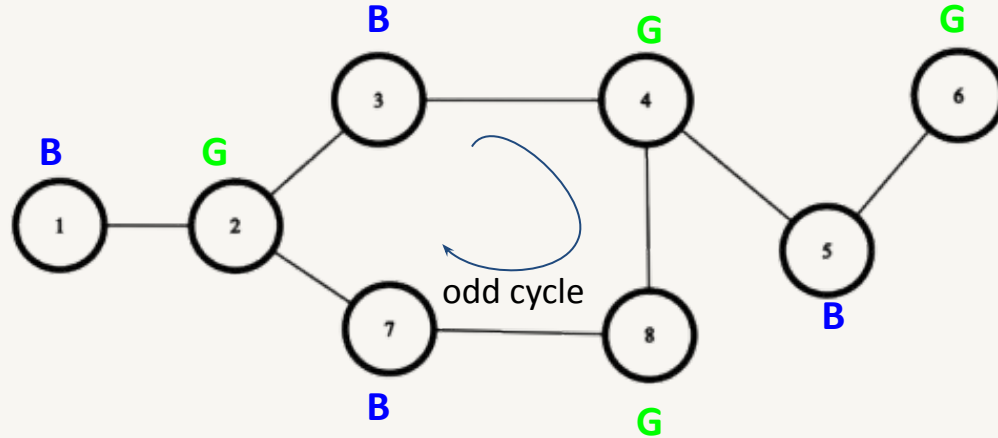
Let's assume two colors,

B -> Blue

G -> Green



Bipartite/Bicolorable Graph



This is not bipartite graph!

Bipartite Graph Detection

- Therefore, it can be said that, any graph that has
 - Odd Cycle -> is not a bipartite graph
 - Even Cycle -> is a bipartite graph
- Similarly,
 - Bipartite Graph -> means it has even cycle
 - Non Bipartite Graph -> means it has odd cycle

Bipartite Graph Detection using BFS

```
Bipartite(G, s)
Q = [], visited = [], color = []
visited[s] = 1, color[s] = "B"
ENQUEUE(Q, s)
while Q is not empty
    u = DEQUEUE(Q)
    for each vertex v ∈ Adj[u]
        if visited[v] = 0 then visited[v] = 1
            if color[u] == "B" then color[v] = "G"
            else color[v] = "B"
            ENQUEUE(Q, v)
        else if color[u] == color[v] return False
return True
```